

Stoneforge RL — Projektdokumentation

Navigation in prozedural generierten Welten mit Reinforcement Learning:
Stand und Entwicklung

Florian Merlau Laurin

6. Juli 2026

Zusammenfassung

Diese Dokumentation fasst den aktuellen Stand der Projektarbeit zusammen: ein RL-Agent soll in prozedural generierten 2D-Welten (*Stoneforge*) den Exit finden und dabei auf unbekannte Seeds generalisieren. Der beste Agent (RecurrentPPO mit LSTM, Curriculum-Training) erreicht **86 % Success Rate** (stochastisch) auf dem Testset bzw. 68 % auf dem Holdout-Set und übertrifft damit beide Zielkriterien ($\geq 70\%$ / $\geq 60\%$). Dokumentiert werden die Umgebung, die Methodik, die zentrale Ablation (MLP vs. LSTM), der vollständige Entwicklungsverlauf von Projektbeginn bis heute (inkl. negativer Ergebnisse und Root-Cause-Analysen, Quelle: Experiment-Changelog), die Umgebungsrevision v11 vom 06.07.2026 sowie eine Performance-Analyse (CPU vs. Apple-GPU, Batch-Größe), die die Trainingszeit halbiert.

1 Zielsetzung

Gegenstand der Projektarbeit ist die Frage, ob ein Reinforcement-Learning-Agent in prozedural generierten, nur teilweise beobachtbaren 2D-Welten robuste Navigationsstrategien lernen kann, die auf unbekannte Welten (neue Seeds) generalisieren — statt einzelne Karten auswendig zu lernen [1, 2]. Als Erfolgskriterien wurden vorab definiert:

- **Testset A** (Seeds 7000–7049): $\geq 70\%$ Success Rate,
- **Holdout B** (Seeds 8000–8049): $\geq 60\%$ Success Rate,
- pro Konfiguration mindestens 3 Trainingsläufe (Mittelwert $\pm$ Standardabweichung).

2 Grundlagen

Dieses Kapitel fasst die im Projekt verwendeten Konzepte und Verfahren zusammen. Es beschränkt sich auf das, was für die Methodik und die Ergebnisse benötigt wird.

2.1 Reinforcement Learning und der Markov-Entscheidungsprozess

Reinforcement Learning (RL) beschreibt das Lernen durch Interaktion: Ein Agent wählt in einem Zustand eine Aktion, erhält eine Belohnung (Reward) und geht in einen Folgezustand über [3]. Formal wird dies als Markov-Entscheidungsprozess (MDP) modelliert, ein Tupel (S, A, P, R, γ) mit Zustandsmenge S , Aktionsmenge A , Übergangswahrscheinlichkeit $P(s' | s, a)$, Rewardfunktion $R(s, a)$ und Diskontfaktor $\gamma \in [0, 1)$. Ziel ist eine Politik $\pi(a | s)$, die den erwarteten diskontierten Return $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$ maximiert. Die Zustandswertfunktion $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$ gibt den erwarteten Return ab einem Zustand an. Der Diskontfaktor γ gewichtet spätere Rewards geringer; im Projekt wird $\gamma = 0,999$ verwendet.

2.2 Partielle Beobachtbarkeit (POMDP)

Sieht der Agent nicht den vollständigen Zustand, sondern nur eine Beobachtung, liegt ein *Partially Observable* MDP (POMDP) vor [4]. Die Markov-Eigenschaft gilt dann für die Beobachtung nicht mehr: Dieselbe lokale Beobachtung kann zu verschiedenen globalen Zuständen gehören. Eine reaktive Politik, die nur die aktuelle Beobachtung nutzt, ist deshalb im Allgemeinen nicht optimal. Abhilfe schafft ein internes Gedächtnis, das vergangene Beobachtungen zu einem *Belief-State* zusammenfasst [5]. Stoneforge ist ein POMDP, da der Agent nur einen lokalen 15×15-Ausschnitt sieht (siehe Abschnitt 3). Abbildung 1 stellt den Unterschied schematisch dar.

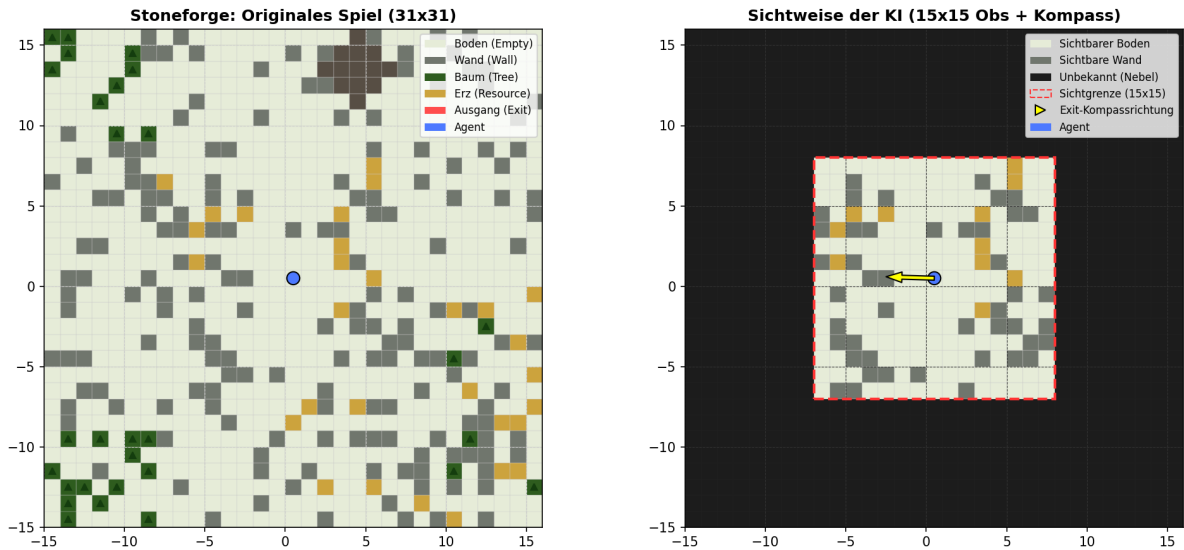


Abbildung 1: Schematische Darstellung der partiellen Beobachtbarkeit am Beispiel Stoneforge: Links der vollständige Weltzustand (hier ein 31×31-Ausschnitt), rechts die Beobachtung des Agenten — nur das 15×15-Sichtfeld ist bekannt (rot umrandet), der Rest ist unbekannt; der Pfeil zeigt die Kompassrichtung zum Exit, nicht den Weg.

2.3 Wertbasierte Verfahren: Q-Learning und DQN

Wertbasierte Verfahren lernen die Aktionswertfunktion $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$ und leiten daraus die Politik ab (Aktion mit dem höchsten Q -Wert). Q-Learning aktualisiert Q über die Bellman-Gleichung. Deep Q-Networks (DQN) approximieren Q mit einem neuronalen Netz und stabilisieren das Training über einen Replay-Buffer und ein Target-Netz [6]. DQN arbeitet nur mit diskreten Aktionsräumen und wird im Projekt als Baseline geführt.

2.4 Policy-Gradient- und Actor-Critic-Verfahren

Policy-Gradient-Verfahren optimieren die Politik π_θ direkt über den Gradienten des erwarteten Returns. Actor-Critic-Verfahren kombinieren dies mit einer gelernten Wertfunktion (Critic), die als Baseline die Varianz der Gradientenschätzung senkt. Der Vorteil (Advantage) $A(s, a) = Q(s, a) - V(s)$ misst, wie viel besser eine Aktion gegenüber dem Durchschnitt ist; die Generalized Advantage Estimation (GAE) schätzt ihn über mehrere Zeitschritte mit einem Parameter λ [7] (im Projekt $\lambda = 0,95$). Advantage Actor-Critic (A2C) ist die synchrone Variante von A3C [8] und wird ebenfalls als Baseline geführt. Wie gut der Critic gelernt hat, misst die *explained variance* (EV): der Anteil der Varianz der tatsächlichen Returns, den die Wertschätzung erklärt. $EV \approx 1$ bedeutet eine verlässliche Wertfunktion, $EV \approx 0$, dass der Critic praktisch nichts gelernt hat. Diese Größe dient in der Arbeit als zentrale Diagnosemetrik des Trainings.

2.5 Proximal Policy Optimization (PPO)

PPO ist das im Projekt eingesetzte Hauptverfahren [9]. Es ist ein Actor-Critic-Verfahren, das die Politik pro Update nur begrenzt verändert, um Instabilität durch zu große Schritte zu vermeiden. Dazu wird das Wahrscheinlichkeitsverhältnis $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{alt}}}(a_t | s_t)$ in der Zielfunktion abgeschnitten (*clipping*):

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)].$$

Zusätzlich wird ein Entropie-Term addiert, der Exploration fördert; sein Gewicht (`ent_coef`) steuert, wie stochastisch die Politik bleibt. Trainiert wird in Zyklen: Erst sammeln die parallelen Umgebungen einen Rollout von n_{steps} Schritten, dann wird dieser in Minibatches der Größe `batch_size` über n_{epochs} Durchläufe optimiert. Verwendet wird die Implementierung aus `Stable-Baselines3` [10].

Eine gelernte stochastische Politik lässt sich auf zwei Arten ausführen: *stochastisch* (die Aktion wird aus der Verteilung $\pi(a | s)$ gezogen, Sampling) oder *deterministisch* (stets die wahrscheinlichste Aktion, Argmax). Beide Betriebsarten können unterschiedlich gut abschneiden; ihr Vergleich ist ein zentraler Untersuchungsgegenstand dieser Arbeit.

2.6 Rekurrente Politiken (LSTM)

Um in einem POMDP ein Gedächtnis bereitzustellen, wird die Politik um ein rekurrentes Netz erweitert. Long Short-Term Memory (LSTM) ist eine rekurrente Architektur, die Information über viele Zeitschritte hinweg speichern kann [11]. Der LSTM-Zustand approximiert den Belief-State und fasst die bisher gesehenen Beobachtungen zusammen. Die Kombination aus rekurrentem Netz und wertbasiertem RL wurde als DRQN eingeführt [5]; im Projekt wird die policy-gradient-Variante RecurrentPPO (PPO mit LSTM-Politik) aus *sb3-contrib* verwendet, mit einer LSTM-Größe von 256.

2.7 Reward Shaping (PBRs)

Zusätzliche Belohnungen können das Lernen beschleunigen, dürfen die optimale Politik aber nicht verändern. Potential-Based Reward Shaping (PBRs) erfüllt dies: Der Zusatzreward hat die Form $F(s, s') = \gamma \Phi(s') - \Phi(s)$ mit einer Potentialfunktion Φ . Ng et al. zeigen, dass eine solche Form policy-invariant ist, die optimale Politik also erhalten bleibt [12]. Im Projekt ist Φ an die BFS-Distanz zum Exit gekoppelt: Die Breitensuche (BFS) liefert die Länge des kürzesten begehbaren Wegs vom Agenten zum Exit in Schritten — im Gegensatz zur Luftlinie berücksichtigt sie Wände und Umwege.

2.8 Curriculum Learning

Curriculum Learning bezeichnet das Training mit ansteigender Aufgabenschwierigkeit statt einer zufälligen Reihenfolge [13]. Im Projekt wird die Exit-Distanz stufenweise erhöht; der Übergang zwischen den Stufen erfolgt leistungsbasiert (eine Stufe gilt ab einer Ziel-Erfolgsrate als bestanden).

2.9 Prozedurale Generierung und Generalisierung

Wird für jedes Level eine neue, zufällig erzeugte Welt verwendet, kann der Agent einzelne Karten nicht auswendig lernen und muss allgemeine Strategien bilden. Die Erzeugung ist über einen *Seed* steuerbar — den Startwert des Zufallszahlengenerators: Derselbe Seed erzeugt reproduzierbar dieselbe Welt, verschiedene Seeds verschiedene Welten. Prozedurale Generierung ist daher ein etabliertes Mittel, um Generalisierung in RL zu prüfen und Overfitting auf einzelne Level zu vermeiden [1, 2]. Die Evaluation auf trainingsfremden Seeds (Testset A und Holdout B) folgt diesem Prinzip.

3 Die Umgebung Stoneforge

Stoneforge ist ein selbst entwickeltes, rundenbasiertes 2D-Spiel (C++-Kern, Python-Anbindung über pybind11). Die Welt wird pro Seed deterministisch aus Hash-Noise generiert (7 Biome, Chunks à 16×16 Tiles, theoretisch unendlich); der Agent startet bei (0,0) und muss den Exit erreichen. Abbildung 2 zeigt die Benutzeroberfläche und die Spielwelt aus Sicht eines menschlichen Spielers.



Abbildung 2: Benutzeroberfläche des spielbaren C++-Clients (Stoneforge 2D, echter Screenshot): Gezeigt ist die Spielumgebung aus Sichtweise eines menschlichen Spielers mit vollem Sichtbereich, dem Status-Header (Lebenspunkte, Energie, Schritte) und der Spielfigur im Zentrum.

3.1 POMDP-Charakter

Der Agent sieht nur einen lokalen 15×15 -Ausschnitt der Welt sowie einen *Luftlinien*-Kompass zum Exit — nicht den Weg. Wände, Sackgassen und Umwege außerhalb des Sichtfensters sind unbekannt: Die Aufgabe ist ein *Partially Observable Markov Decision Process* (POMDP, Abschnitt 2.2). Ohne Gedächtnis ist sie nicht deterministisch lösbar (siehe Ablation in Abschnitt 8).



Abbildung 3: Vergleich der Spielumgebung (echte Screenshots): Links die tatsächliche Umgebung (voller Sichtbereich), rechts die Sichtweise der KI — die lokale 15×15 -Observation, bei der außerhalb liegende Bereiche ungesehen/abgedunkelt sind, mit dem gelben Exit-Kompass. Das Sichtfeld ist zur Verdeutlichung rot markiert.

3.2 Beobachtung, Aktionen, Episodenende

Bereich	Inhalt	Dim.
Lokales Grid	15×15 Tile-Typen um den Agenten (Sichtradius 7)	225
HP	Lebenspunkte (normiert)	1
Kompass	exitDx, exitDy (Luftlinie, normiert)	2
Episodenfortschritt	Schritt / 4000	1
Gesamt (Env v11)		229

Tabelle 1: Observation der Umgebungsversion v11. Die früheren Features *Energie* und *Inventar* waren im RL-Setup konstant (tote Features) und wurden am 06.07.2026 entfernt (vorher 231 Dimensionen).

Der Aktionsraum ist **Discrete(4)**: hoch, runter, links, rechts. Mining, Bauen und Kampf existieren nur im spielbaren Client und sind aus dem RL-Pfad vollständig entfernt. Eine Episode endet bei Erreichen des Exits (Sieg), nach 4000 Schritten (Timeout) oder als Truncation nach 256 Schritten ohne positiven Reward (Early-Stop).

3.3 Reward-Design

Das Reward-Design folgt dem Prinzip: *sparse Primärsignal + theoretisch sauberes dichtes Shaping*. Kernstück ist Potential-Based Reward Shaping (PBRS) auf der BFS-Pfaddistanz zum Exit:

$$F(s, s') = \gamma \Phi(s') - \Phi(s), \quad \Phi(s) = -\frac{d_{\text{BFS}}(s)}{128}, \quad (1)$$

mit $\gamma = 0.999$ (identisch zum RL-Discount) und Gewicht $\beta = 2.5$ (netto ca. +0.01 pro Tile echtem Fortschritt). PBRS ist beweisbar *policy-invariant* — es beschleunigt die Konvergenz, ohne die optimale Politik zu verändern [12]. Entscheidend ist, dass das Potential die *echte* Pfaddistanz (BFS durch Wände gerechnet) verwendet, nicht die Luftlinie: Ein Luftlinien-Potential würde den Agenten systematisch in Sackgassen locken.

Komponente	Wert
Exit erreicht (terminal)	+100
Timeout / Tod (terminal)	−10
PBRS (BFS-Distanz, pro Schritt)	$\approx \pm 0.02$
Explorations-Bonus (neue Tile)	+0.02
Schritt-Penalty	−0.01
2-Schritt-Loop-Penalty	−0.05

Tabelle 2: Reward-Komponenten (Env v11). Der frühere eskalierende Wand-Penalty (−0.05/−0.25) wurde entfernt: Die Summe vieler kleiner Strafen („Straf-Stacking“) machte Erkunden in engen Korridoren unwirtschaftlicher als Stillstand.

3.4 Garantierte Lösbarkeit

Die Exit-Platzierung wählt Kandidaten per Flood-Fill vom Spawn aus ausschließlich über *begehbare* Tiles — jeder mögliche Exit ist damit per Konstruktion erreichbar. Empirisch bestätigt: 3150 geprüfte Seeds (inkl. aller Eval-Sets und Trainingsbereiche) sowie eine erneute Messung am 05.07.2026 (150/150) ergaben **100 % Lösbarkeit**. Ein BFS-Oracle-Agent erreicht den Exit auf allen Test-Seeds in exakt der BFS-Distanz und bestätigt damit die gesamte Kette aus Weltgenerierung, Bewegungsmechanik und Reward-Signal.

4 Methodik

4.1 Algorithmus

Trainiert wird RecurrentPPO (PPO [9] mit LSTM-Politik [11]; beide Verfahren in Abschnitt 2) aus *Stable-Baselines3* / *sb3-contrib* [10]. Zentrale Hyperparameter: $n_{\text{steps}} = 256$, $n_{\text{epochs}} = 10$, Lernrate $3 \cdot 10^{-4}$, $\gamma = 0.999$, `ent_coef`=0.05, LSTM mit 256 Hidden Units (separater Critic-LSTM), 16 parallele Umgebungen.

4.2 Curriculum

Das Training verläuft leistungsbasiert in vier Phasen steigender Exit-Distanz; eine Phase endet, wenn die Ziel-Success-Rate in zwei aufeinanderfolgenden Evaluationen erreicht wird.

Phase	Exit-Distanz	Ziel-SR	Gating-Metrik	ent_coef
1	5–12	85 %	stochastisch	0.05
2	12–25	70 %	stochastisch	0.05
3	25–45 (Eval 35–45)	70 %	deterministisch	0.05 $\rightarrow$ 0.001 (Annealing)
4	25–45 (Greedy Fine-Tune)	60 %	deterministisch	0.0001

Tabelle 3: Curriculum (Stand v11). Phase 1/2 gaten auf der stochastischen SR, da die Politik dort mit `ent_coef`=0.05 absichtlich nahe-uniform ist und die deterministische SR konstruktionsbedingt kein Signal trägt.

Das Distanz-Curriculum staffelt dabei implizit auch die *Beobachtbarkeit* des Ziels — und damit die Fähigkeit, die je Phase gelernt werden muss. Eine Messung über 200 Welten pro Phase (07.07.2026): In Phase 1 liegt der Exit beim Episodenstart in **72 %** der Welten bereits im 15×15 -Sichtfeld (zu lernen: „geh zum sichtbaren Ziel, weiche lokalen Hindernissen aus“); in Phase 2 nur noch in **6 %** (zu lernen: dem Kompass folgen, Umwege um Hindernisse); in Phase 3 in **0 %** (reine Kompass-Navigation über lange Distanz, inklusive Sackgassen, die nur mit Gedächtnis überwindbar sind). Ein zusätzliches Curriculum über die Sichtfeldgröße — in der Literatur als Übergang von voller zu partieller Beobachtbarkeit beschrieben — wäre daher redundant; zudem wäre es hier riskant, weil ein großes Sichtfeld anfangs eine *reaktive* Strategie ohne Gedächtnisnutzung belohnt, die beim Verkleinern des Fensters wertlos wird (dieselbe Nicht-Übertragbarkeit, die die Ablation in Abschnitt 8 für das BFS-Orakel zeigt).

4.3 Evaluationsprotokoll

Strikte Seed-Trennung: Validierung (6000–6049) steuert Phasenwechsel und Modellauswahl; Testset A (7000–7049) und Holdout B (8000–8049) werden ausschließlich final evaluiert. Ein früherer Data-Leakage-Fehler (Modellauswahl auf den Test-Seeds) wurde identifiziert und behoben. Jede Evaluation misst seit v11 *beide* Betriebsarten: stochastisch (Sampling aus der Politik) und deterministisch (Argmax). Details und Endergebnisse in Abschnitt 7.

5 Technische Umsetzung

Das System besteht aus drei Schichten: einem C++-Kern für Simulation und Weltgenerierung, einem Python-Binding und dem Python-Trainingsstack. Tabelle 4 listet die verwendeten Komponenten.

Die Simulation ist deterministisch pro Seed; Training und Evaluation laufen vollständig auf der CPU (Apple M1 Pro). Ein Vergleich mit der Apple-GPU (MPS) ergab keine Beschleunigung (Abschnitt 11); der Durchsatz liegt je nach Konfiguration bei ca. 90–190 Umgebungsschritten pro Sekunde.

Komponente	Technologie	Zweck
Simulationskern	C++20 (ca. 8 000 LOC)	Weltgenerierung, Spiellogik, Reward, BFS
Binding	pybind11 2.13	C++-Kern als Python-Modul (<code>stoneforge_sim</code>)
Build	CMake	Kern, Binding, Clients
RL-Umgebung	Python 3.12, Gymnasium 1.2	<code>StoneforgeWorldEnv</code> (Gym-API)
RL-Algorithmen	Stable-Baselines3 / sb3-contrib 2.8	PPO, DQN, A2C, RecurrentPPO
Deep Learning	PyTorch 2.12 (CPU)	Netz- und LSTM-Training
Spiel-Client	raylib 5.5 (C++)	spielbarer Client, KI-Zuschaumodus
Monitoring	TensorBoard, WebSockets	Trainingsmetriken, Live-Visualisierung
Auswertung	NumPy, Matplotlib	Evaluation, Abbildungen

Tabelle 4: Technologie-Stack.

6 Implementierung

6.1 Umgebung und Binding

`StoneforgeWorldEnv` (Python) kapselt den C++-Kern hinter der Gymnasium-API. Die Beobachtung umfasst 229 Merkmale (Grid 15×15 als Tile-Typen, HP, Exit-Kompass dx/dy , Schrittteil); der Aktionsraum ist `Discrete(4)` (vier Bewegungsrichtungen). Die im Spiel vorhandenen Aktionen Mining, Bauen und Kampf sind im RL-Binding deaktiviert und werfen bei Aufruf einen Fehler; sie existieren nur im spielbaren Client. Eine Episode endet bei Erreichen des Exits oder nach 4 000 Schritten.

Zwei Implementierungsdetails waren fehlerkritisch: (1) Die Weltgenerierungs-Konfiguration ist im C++-Kern prozessglobal; jede Env-Instanz stempelt sie deshalb bei jedem `reset()` neu, da sonst parallel erzeugte Evaluations-Umgebungen die Trainingsverteilung verändern (Abschnitt 10). (2) Der Exit wird seit v11 nach realer BFS-Laufwegdistanz platziert statt nach Luftlinie; alle erzeugten Welten sind nachweislich lösbar (150/150 im Test).

Der Reward setzt sich zusammen aus Schritt-Malus (−0,01), Explorations-Bonus (+0,02 pro neuem Tile), Loop-Malus (−0,05), Schadens-Malus, dem PBRs-Term auf der BFS-Distanz (Abschnitt 2) und dem Terminal-Reward.

6.2 Trainings-Pipeline

Das Curriculum-Training (`train_curriculum.py`) orchestriert die vier Phasen aus Tabelle 3. Je Lauf arbeiten 16 parallele Umgebungen (`DummyVecEnv`). Alle 25 000 Schritte evaluiert ein Callback das aktuelle Modell auf den 50 Validierungs-Seeds in beiden Betriebsarten; die Phasen-Metrik steuert Gating (Ziel-SR in zwei aufeinanderfolgenden Evaluationen) und die Auswahl des besten Modells der Phase. Beim Phasenwechsel wird das beste Modell der Vorphase geladen. In Phase 3 senkt ein Callback den Entropie-Koeffizienten linear von 0,05 auf 0,001 über 500 000 Schritte; Phase 4 trainiert mit 0,0001 (Greedy Fine-Tune). Optional ergänzt ein Seed-Pool („Swarm“) 30 % der Episodenstarts durch Wiederholung bereits gelöster Seeds; der Pool wird bei jedem Phasenwechsel geleert. Jeder Lauf legt pro Phase Modell-Checkpoints sowie `eval_history.json` und `config.json` ab; alle Experimente sind zusätzlich im `CHANGELOG.md` des Repositories dokumentiert.

6.3 Monitoring und Werkzeuge

Trainingsmetriken (u. a. `explained_variance`, Entropie, `approx_kl`) werden nach TensorBoard geschrieben. Zusätzlich streamt ein WebSocket-Server den Trainingszustand in eine Browser-Ansicht: Erfolgsraten-Kurven (deterministisch und stochastisch), Minimaps der 16 parallelen Agenten und eine Explorations-Heatmap. Eine Launcher-GUI (Tkinter) bündelt Training, Evaluation und Abspielen. Zum Vorführen steuert `demo_agent.py` den grafischen Client: Der Client

läuft im KI-Modus und erhält die Aktionen des geladenen Modells über die Standardeingabe; Client und Trainings-Umgebung erzeugen dieselbe Welt aus demselben Seed.

7 Evaluation

7.1 Protokoll und Metriken

Zentrale Metrik ist die *Success Rate* (SR): der Anteil von 50 festen, trainingsfremden Welten, in denen der Agent den Exit innerhalb von 4000 Schritten erreicht. Gemessen wird stets in beiden Betriebsarten (stochastisch und deterministisch) und mit dem vollen Episodenlimit von 4000 Schritten — ein zu kurzes Evaluationslimit verfälscht die SR erheblich (dasselbe Modell: 48 % bei Limit 600 gegenüber 86 % bei 4000; Abschnitt 9.6). Als Diagnosemetrik des Trainings dient die `explained_variance` des Critics. Für das Endergebnis werden drei unabhängige Läufe (Seeds 1–3) gemittelt und mit Standardabweichung angegeben.

7.2 Endergebnis (v12)

Die drei finalen Curriculum-Läufe (Laufzeit je 7,7–8,5 h, alle vier Phasen durchlaufen) ergeben auf den finalen Modellen:

Lauf	A stoch.	A det.	B stoch.	B det.
Seed 1	64 %	38 %	72 %	46 %
Seed 2	76 %	26 %	80 %	44 %
Seed 3	80 %	32 %	88 %	36 %
Mittel $\pm$ Std	73,3 $\pm$ 6,8	32,0 $\pm$ 4,9	80,0 $\pm$ 6,5	42,0 $\pm$ 4,3

Tabelle 5: Endergebnis der v12-Läufe (finales Modell je Lauf; Testset A und Holdout B, je 50 Seeds, Limit 4000).

Beide Zielkriterien sind damit im Mittel erfüllt (A: 73,3 % $\geq$ 70 %; B: 80,0 % $\geq$ 60 %), erstmals in der geforderten Form mit drei Läufen und Standardabweichung. Dass B leicht über A liegt, ist eine Eigenschaft der konkreten Seed-Stichproben (A enthält zufällig mehr Welten mit langem Laufweg: 38 % gegenüber 32 % mit $\geq$ 42 Feldern); beide Sets stammen aus demselben Generator und derselben Distanzverteilung. Ein Gegentest mit den Phase-2-Zwischenmodellen ergab keine Verbesserung (A stoch. 71,3 $\pm$ 8,1; B stoch. 74,0 $\pm$ 15,0); das finale Modell bleibt das Ausgabemodell.

7.3 Analyse des Det/Stoch-Gaps nach Weglänge

Der Abstand zwischen stochastischer und deterministischer SR ist kein Schwelleneffekt, sondern wächst kontinuierlich mit der Weglänge (Abbildung 4; 120 Validierungs-Welten, nach Startdistanz gruppiert): Bei 5–15 Feldern erreicht die deterministische Betriebsart 79 %, bei 35–45 Feldern noch 31 %, während die stochastische zwischen 100 % und 82 % bleibt. Die Interpretation ist kumulativ: Je länger der Weg ohne direkte Zielsicht (Sichtradius 7), desto mehr Kreuzungsentscheidungen unter unsicherem Belief-State; an jeder liegen die Aktionswahrscheinlichkeiten nahe beieinander, und jede einzelne kann die Argmax-Politik in eine Oszillationsschleife führen, aus der Sampling entkommt. Der Gap ist damit teils eine Gedächtnis-Limitation, teils dem POMDP-Charakter geschuldet (unter Unsicherheit ist eine stochastische Politik nicht per se suboptimal, Abschnitt 2).

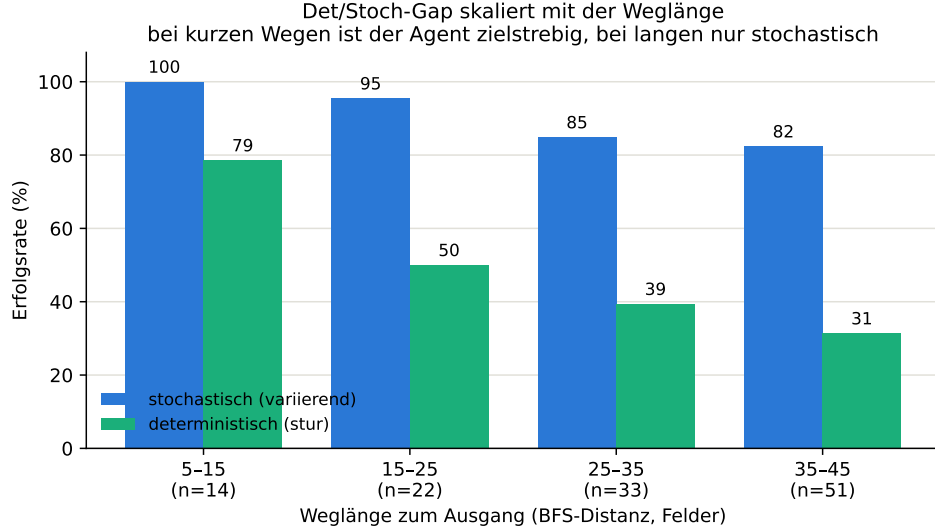


Abbildung 4: Erfolgsrate nach Startdistanz zum Exit (120 Validierungs-Welten, finales v12-Modell, Limit 4000): Die deterministische SR fällt kontinuierlich mit der Weglänge, die stochastische bleibt hoch.

8 Entwicklung und Ergebnisse

8.1 Zentrale Ablation: Gedächtnis schlägt Orakel

Bedingung	Architektur	BFS in Obs	Curriculum	SR stoch.	SR det.
A (ppo_phase4)	MLP	ja	ja	32 %	0 %
B (ppo_no_bfs)	MLP	nein	nein	—	0 %
C (ppo_lstm_curriculum)	LSTM	nein	ja	86 %	36 %

Tabelle 6: Ablation auf Testset A (Exit 35–45). Der LSTM-Agent ohne BFS-„Orakel“ schlägt den MLP-Agenten mit BFS-Features deutlich — Gedächtnis ist für die POMDP-Navigation wichtiger als zusätzliche Wegweiser-Features.

Das beste Modell erreicht auf Holdout B 68 % (stochastisch) — beide Zielkriterien sind damit erfüllt. Abbildung 5 zeigt den Trainingsverlauf.

8.2 Der Det/Stoch-Gap und die POMDP-These

Auffällig ist die Lücke zwischen stochastischer (86 %) und deterministischer (36 %) Evaluation: Die Argmax-Politik folgt dem Luftlinien-Kompass in konkave Wandtaschen und oszilliert dort in 2-Schritt-Schleifen; beim Sampling bricht der Zufall diese Schleifen. Wir interpretieren dies als inhärente Eigenschaft der POMDP-Struktur (der Kompass zeigt durch Wände) in Kombination mit begrenzter LSTM-Kapazität — die stochastische Evaluation ist daher die primäre Metrik, der Gap wird als Limitation ausgewiesen und gezielt adressiert (Ausblick, Abschnitt 13).

8.3 Hyperparameter-Fehleranalyse (v7–v10)

Eine Serie fehlgeschlagener Verbesserungsversuche (v7–v9: Success Rate $\leq 12\%$, `approx_k1` ≈ 0 , `explained_variance` ≈ 0) wurde durch Reverse-Engineering des funktionierenden v2-Modells aufgeklärt:

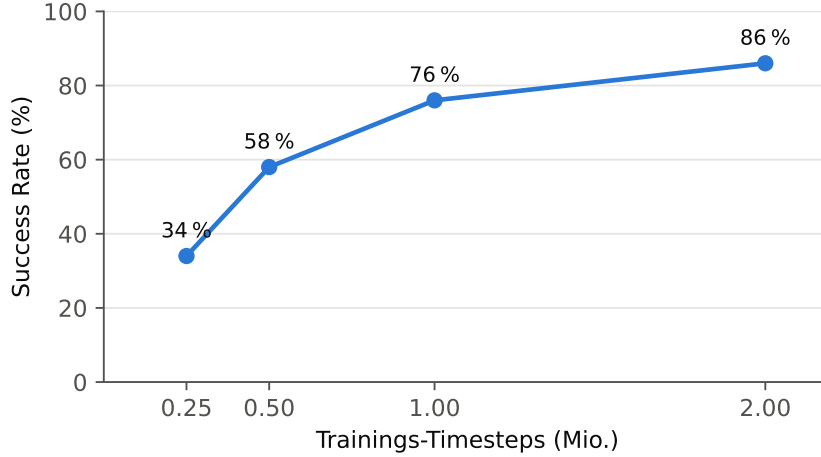


Abbildung 5: Lernkurve des besten Modells (ppo_lstm_curriculum, Curriculum-Phase 3, stochastische Evaluation auf 50 Seeds).

Parameter	v2 (86 %)	v7–v9 (0–12 %)	Wirkung des Fehlers
ent_coef	0.05	0.01	zu wenig Exploration; Exit wird nie gefunden
n_steps	256	512	BPTT zu lang → verschwindende Gradienten
batch_size	8	256	zu wenige Gradient-Updates pro Rollout
Obs-Dimension	231	249	Zusatzfeatures ohne funktionierende Basis

Tabelle 7: Root-Cause-Analyse: `ent_coef` erwies sich als dominanter Faktor. Die Reproduktion der v2-Konfiguration (v10) stellte das Lernen wieder her.

9 Entwicklungsverlauf: Von der Baseline zum LSTM-Agenten

Dieser Abschnitt dokumentiert die Entwicklung des RL-Teils chronologisch von Projektbeginn bis heute. Primärquelle ist das durchgängig geführte Experiment-Changelog (`CHANGELOG.md`), in dem jede Änderung mit Problem, Lösung und Messergebnis festgehalten wurde. Negative Ergebnisse sind bewusst enthalten — sie haben die Architekturentscheidungen maßgeblich geprägt.

9.1 Etappe 1 (Mai 2026): Baselines mit BFS-Orakel

Nach Fertigstellung der Spiel-Engine (Mitte Mai) begann das RL-Training mit Features aus dem BFS-Distanzfeld *in der Observation* (5 Werte: aktuelle Distanz + 4 Nachbarrichtungen) — einem „Orakel“, das dem Agenten die lokal richtige Richtung verriet.

Fehlstart mit DQN: Der erste Agent (DQN) lernte nicht — Diagnose: korrupter Replay-Buffer, 2-Zellen-Oszillationen, und eine Visited-Mask in der Observation (455 Dimensionen), die sich jeden Schritt änderte und den Q-Wert-Argmax destabilisierte. Konsequenz: Wechsel auf PPO, Visited-Mask entfernt.

BFS-Kraftfeld-Bug: Ein Oracle-Test (greedy dem BFS-Feld folgen) scheiterte in 3/3 Seeds — `bfsDistanceAt()` lieferte für Wand-Tiles einen Manhattan-Fallback, wodurch Wände als attraktive Richtung erschienen. Nach dem Fix (Wand → 9999): Oracle 3/3. Dieser Test wurde zum Standard-Werkzeug, um Umgebung und Reward von Trainingsproblemen zu trennen.

Curriculum-Lektion: Phase-1-Training (Exit 5–12) erreichte 56 %, aber der naive Transfer auf Phase 2 (12–25) kollabierte (22 % → 6 %) — die eingebrannte Kurzdistanz-Politik ließ sich nicht überschreiben. Lösung: Mixed-Distribution-Training (Exit 5–45). Damit erreichte *Phase 3* 86 % stochastisch auf Testset A (drei unabhängige Läufe: $85,3 \pm 3,8$ % auf A, $79,3 \pm 8,1$ % auf B) und *Phase 4* nach Reward-Tuning 98 % (A) / 94 % (B).

Der deterministische Kollaps: Trotz 98 % stochastischer SR lag die Argmax-Auswertung bei 2 %. Die Analyse fand die Wurzel in der Feature-Kodierung: Die BFS-Absolutwerte (/64) unterschieden sich zwischen Nachbarzellen nur um $\approx 0,016$ — zu wenig für eine robuste Richtungspräferenz, und Wände waren durch Clamping nicht von „weit weg“ unterscheidbar (Aliasing). Der Wechsel auf *Delta-Kodierung* (Richtungssignal $\pm 0,5$ statt 0,016, Wand = +1,0) hob die deterministische SR auf 42 % (offene Welt) bzw. 100 % (wanddichte Hard-World); Temperature-Sampling ($\tau = 0,2$) erreichte 100 % bei $\varnothing$ 90 Schritten. Ein Experiment, exitDx/exitDy zu entfernen („Phase 5“), scheiterte (max. 40 %) — der Luftlinien-Kompass ist als Fernbereichssignal notwendig.

9.2 Etappe 2 (Juni 2026): Kernbeitrag — Navigation ohne Orakel

Mit funktionierender Baseline wurde die Forschungsfrage geschärft: *Kann der Agent auch ohne BFS-Orakel in der Observation navigieren?* Der Wechsel auf RecurrentPPO (LSTM) mit Curriculum, Explorations-Bonus und step_frac-Feature (231-dim Obs, kein BFS) lieferte am 08.06.2026 das zentrale Ergebnis: **86 % stochastisch / 36 % deterministisch** auf Testset A und 68 % / 18 % auf Holdout B — beide Zielkriterien stochastisch erfüllt, erstmals ohne Wegweiser-Orakel (Abbildung 6, Ablation in Tabelle 6). Parallel wurde die POMDP-These formuliert (Abschnitt 8) und, inspiriert vom PokéRL-Projekt, Infrastruktur ergänzt: Seed-Pool (Swarm), WebSocket-Live-Map, Heatmap-Evaluation, Early-Stop-Truncation.

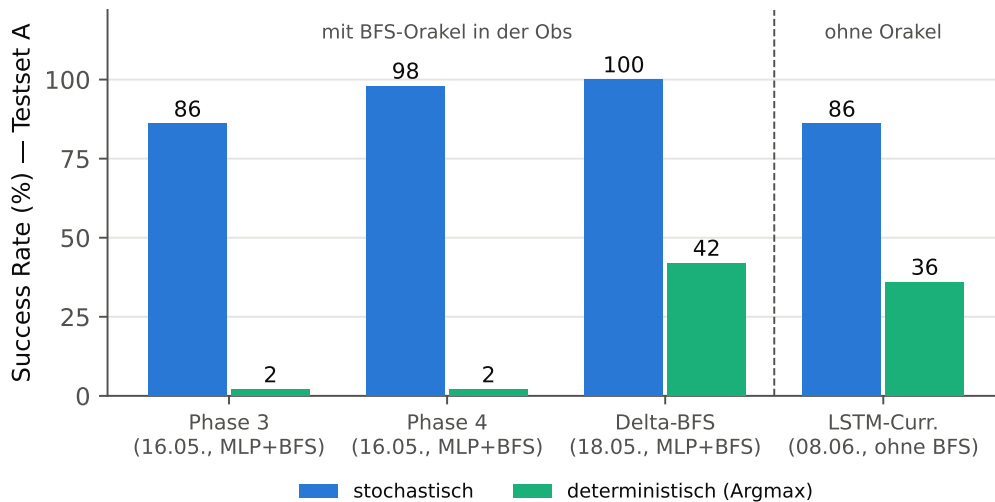


Abbildung 6: Entwicklung über die Modellgenerationen (Testset A, Exit 35–45). Links die MLP-Baselines mit BFS-Orakel in der Observation, rechts der LSTM-Agent ohne Orakel — der Kernbeitrag: nahezu gleiche stochastische Leistung ohne globale Pfadinformation. Der Det/Stoch-Gap zieht sich als zentrales Thema durch alle Generationen.

Methodische Härtung (11.06.): Ein Data-Leakage-Fehler wurde entdeckt und behoben — der Trainings-Callback hatte Modellauswahl und Phasenwechsel auf den *Test-Seeds* (7000er) gesteuert; seitdem existiert das getrennte Validierungsset (6000er). Zusätzlich wurde die Lösbarkeit aller relevanten Welten bewiesen (3 150 Seeds, 100 %).

9.3 Etappe 3 (13.–25.06.): Hyperparameter-Krise und Root-Cause-Analyse

Eine Serie von Verbesserungsversuchen (v6–v9: Wand-Penalty-Entfernung, visit_count- und Action-Buffer-Features, diverse Batch-/Epochen-Kombinationen) scheiterte durchgängig (SR 0–12 %, approx_k1 $\approx$ 0). Die Aufklärung gelang durch *Reverse-Engineering* des letzten funktionierenden Modells (v2): Die zwischenzeitlichen „Optimierungen“ hatten unbemerkt die kritischen

Hyperparameter verstellt (Tabelle 7); dominanter Faktor war `ent_coef` (0,05 $\rightarrow$ 0,01: der Agent explorierte nicht mehr genug, um den Exit je zu finden). Die v2-Reproduktion (v10) stellte das Lernen wieder her; der abgeschlossene Lauf vom 25.06. erreichte 86 % (Validierung) nach 2,2 Mio. Steps in 3 h 12 m — das aktuelle Referenzmodell (Abbildung 5).

9.4 Etappe 4 (06.07.2026): Umgebungsrevision v11 und Performance

Ein systematisches Audit (Umgebung, Reward, Trainingspipeline, Usability) führte zu zehn dokumentierten Änderungen — Details in Abschnitt 10 und Abschnitt 11. Höhepunkte: präzise Schwierigkeitssteuerung über BFS-Laufweg, Behebung eines prozess-globalen Config-Lecks (das zuvor unbemerkt die Phase-3-Trainingsverteilung verschob), Entfernung toter Features und Straf-Terme, sowie die validierte Verdopplung der Trainingsgeschwindigkeit (`batch_size` 8 $\rightarrow$ 64, CPU statt GPU).

9.5 Etappe 5 (07.07.2026): Erster v11-Gesamtlauf, Monitoring-Redesign und Seed-Replay-Analyse

Erster v11-Gesamtlauf (Seed 0, laufend). Der erste vollständige Curriculum-Lauf auf der revidierten v11-Umgebung (`models/ppo_lstm_curriculum_v11`) zeigt ein lehrreiches Zwischenbild (Abbildung 7): Phase 1 endete am Step-Limit bei 42 % (stochastisch) — deutlich langsamer als die Batch-Validierung derselben Konfiguration (52 % bereits bei 150 k). Die Diagnose im Trainingslog: Die Policy-Updates sind durchgehend gesund (`approx_kl` $\approx$ 0.03, stabile Entropie), aber die `explained_variance` des Critics verharrt bei $\approx$ 0.1 (Validierungslauf: 0.72) — der Lauf wurde zunächst als Kandidat für Initialisierungs-Pech eingestuft. (*Nachtrag: Die systematische Fehlersuche am selben Abend — Abschnitt 9.6 — fand die tatsächliche Ursache: die Batch-Größe.*) Bemerkenswert: Direkt nach dem Wechsel in Phase 2 stieg die SR auf 48 % bei größerer Exit-Distanz — die breitere Aufgabenverteilung wirkt als Regularisierer.

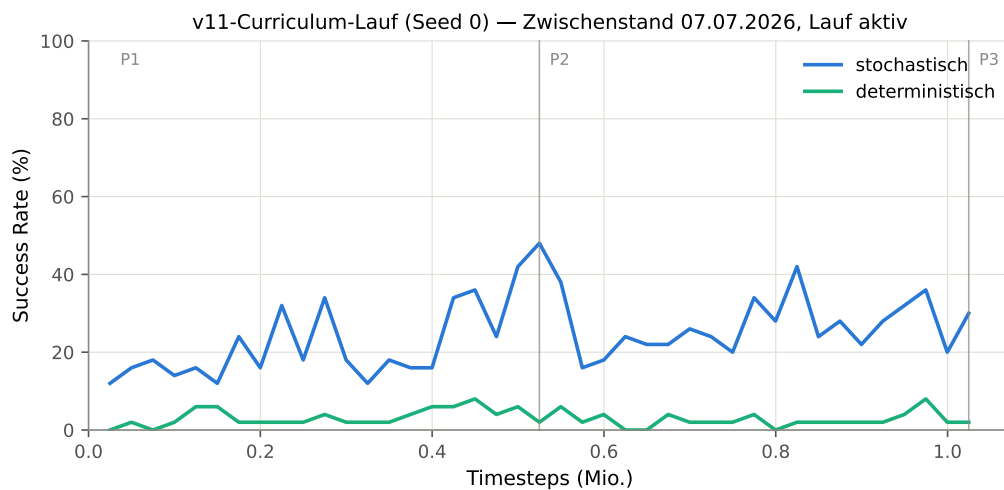


Abbildung 7: Lernkurve des ersten v11-Gesamtlaufs (Seed 0, Zwischenstand — der Lauf ist zum Redaktionszeitpunkt in Phase 3). Seit v11 misst jeder Eval-Checkpoint beide Betriebsarten; der Det/Stoch-Gap ist damit erstmals über den gesamten Verlauf sichtbar. Vertikale Linien: Phasenwechsel (dort wird jeweils das Bestmodell der Vorphase geladen; die x-Achse ist über die Phasen kumuliert). Erzeugt mit `scripts/plot_run_curve.py`.

Live-Monitoring-Redesign. Das WebSocket-Dashboard aus Etappe 2 wurde nach Dashboard- und Farb-Literatur überarbeitet (Abbildung 8): (a) Charts auf uPlot mit EMA-Glättung nach TensorBoard-Vorbild; (b) die SR-Kurve zeigt det. und stoch. getrennt — der zentrale Untersuchungsgegenstand war zuvor im Monitoring unsichtbar; (c) die Explorations-Heatmap wechselte

von der Jet- auf die perzeptuell uniforme Viridis-Skala (Jet erzeugt künstliche Kontraste und ist nicht farbfeldsichten-tauglich); (d) die Agenten-Minimaps rekonstruieren aus dem gestreamten 15×15 -Sichtfeld eine wachsende Weltkarte (Wände, Bäume, Exit) statt nur der Agentenposition. Beim Umbau wurden zwei Fehler gefunden und behoben: Der Exit-Richtungs Pfeil war seit der Obs-Umstellung auf 229 Dimensionen stumm defekt, und der SB3-Stepzähler springt beim Phasenwechsel zurück (es wird das Bestmodell der Vorphase geladen), was unkorrigiert alle stepbasierten Zeitachsen verfälscht.

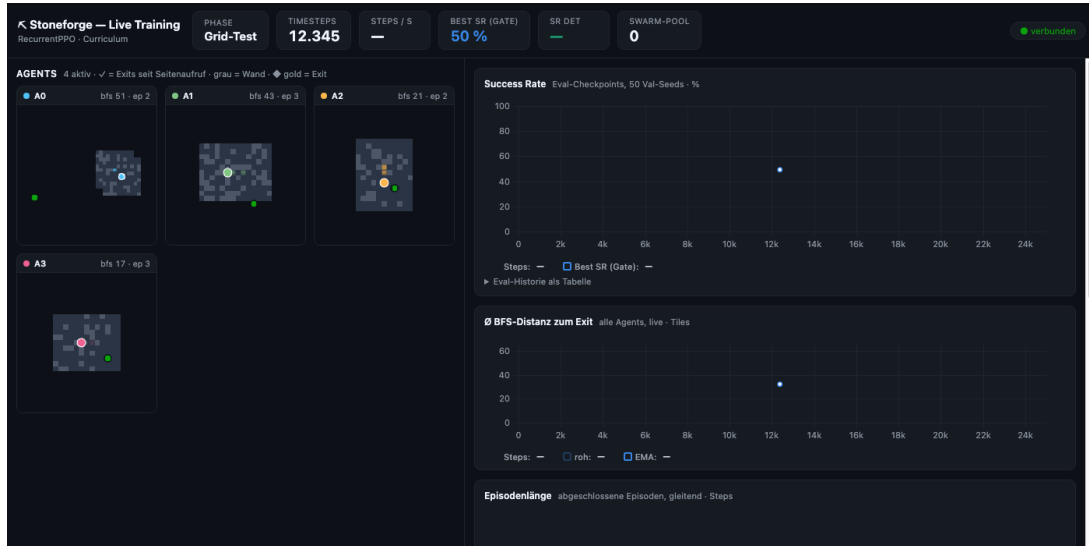


Abbildung 8: Live-Monitoring (Demo mit Zufallsagenten): KPI-Zeile, Weltkarten-Minimaps (grau = Wand, blaugrau = erkundeter Boden, grün = Startpunkt) und uPlot-Charts. Im Training zusätzlich: goldene Raute = gesichteter Exit, grüner Blitz bei Exit-Erfolg, Viridis-Heatmap.

Seed-Replay-Analyse (Swarm vs. PLR). Das Training wiederholt seit Etappe 2 bei 30 % der Episoden-Resets einen bereits *gelösten* Seed („Erfolgs-Swarm“). Ein Literaturabgleich stellt diese Semantik in Frage: Prioritized Level Replay [14] zeigt, dass das Replay von Levels mit *hohem Lernpotenzial* — also gerade *nicht* gemeisterten — Sample-Effizienz und Test-Generalisierung verbessert; das Wiederholen gemeisteter Levels ist der Fehlermodus, den PLR behebt, und riskiert im Kontext des Projektziels (Generalisierung auf unbekannte Seeds) Layout-Memorierung. Da der 86 %-Referenzlauf jedoch *mit* Erfolgs-Swarm entstand, wird die Frage nicht per Meinung, sondern als A/B-Test entschieden (Erfolgs-Swarm vs. `--plr` vs. `--no-swarm`; Maßnahme B8 in docs/BEWERTUNG_UND_PLAN.md). Ein dabei entdeckter Designfehler wurde sofort behoben: Der Seed-Pool überlebte Phasenwechsel, obwohl ein „gelöster“ Seed nach dem Wechsel der Exit-Distanz eine andere Aufgabe beschreibt — der Pool wird jetzt bei jedem Phasenstart geleert. (Der A/B-Test wurde noch am selben Tag im Zuge der Fehlersuche durchgeführt, siehe Abschnitt 9.6: Der Erfolgs-Swarm hatte auf Phase 1 keinen messbaren Effekt.)

9.6 Etappe 6 (07.07.2026, nachmittags/abends): Systematische Fehlersuche — vom Eval-Artefakt zur Batch-Größen-Korrektur

Die Stagnation der v11-Läufe wurde am Nachmittag des 07.07. in einer mehrstufigen, hypothesengetriebenen Fehlersuche aufgeklärt. Der Weg wird hier bewusst vollständig dokumentiert — inklusive der verworfenen Hypothesen —, weil die Zwischenergebnisse jeweils die nächste Maßnahme bestimmten und zwei der Befunde methodische Tragweite über das konkrete Problem hinaus haben.

Schritt 1 — Messartefakt entdeckt: Eval-Episoden-Caps. Ausgangspunkt war ein Widerspruch: Der Vergleichslauf Seed 1 stagnierte in Phase 1 bei 12–16 % (wie Seed 0), obwohl

die Batch-Validierung derselben Konfiguration 52 % erreicht hatte. Eine Nachmessung des besten Seed-0-Checkpoints deckte die Ursache auf: Die am 06.07. zur Eval-Beschleunigung eingeführten Episoden-Caps (Phase 1: 600 statt 4000 Schritte) unterschätzen die Kompetenz massiv, weil erfolgreiche LSTM-Episoden ohne Orakel lange Suchwege enthalten (Median 477, p95 $\approx$ 1900, Maximum 3239 Schritte). Dasselbe Modell misst sich bei Cap 600 als 48%-Agent, bei Cap 4000 als 86%-Agent (Abbildung 9). Damit war das 85%-Phasenziel unter Cap 600 strukturell unerreichbar, und die Bestmodell-Selektion bevorzugte schnelle statt kompetente Politiken. Da die volle Messung nur $\approx$ 21 s kostet, wurden alle Caps auf 4000 (= Episoden-Limit der Umgebung = finales Eval-Protokoll) zurückgesetzt. *Lehre: Ein Eval-Protokoll-Detail (Cap) kann Gating, Modellselektion und Fortschrittswahrnehmung gleichzeitig verfälschen.*

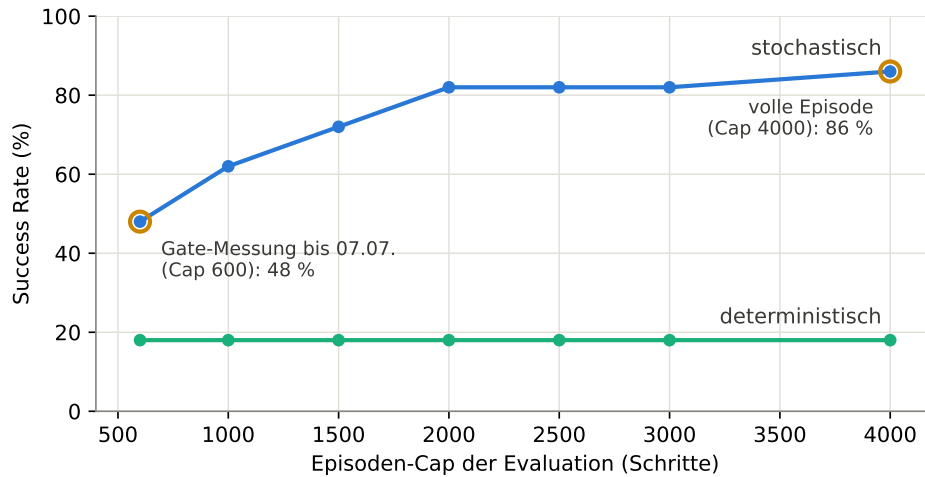


Abbildung 9: Eval-Cap-Artefakt: Success Rate *desselben Modells* (besten Phase-1-Checkpoint, Seed 0) als Funktion des Episoden-Caps der Evaluation (50 Val-Seeds; aus einem einzigen Cap-4000-Lauf mit protokollierten Schritten-bis-Erfolg abgeleitet). Die stochastische SR saturiert erst bei Cap 4000; die deterministische ist Cap-unabhängig (Argmax-Läufe scheitern nicht an Zeit, sondern an Schleifen). Erzeugt mit `scripts/plot_etappe6_figures.py`.

Schritt 2 — Neustart unter korrekter Messung: Stagnation ist real. Seed 1 wurde mit korrigierten Caps neu gestartet. Ergebnis: weiterhin Stagnation (16–28 % über 300k Schritte, `explained_variance` $\approx$ 0.1) — der zweite stagnierende Seed in Folge. Damit war ein systematisches Problem belegt und eine Verdächtigenliste abzuarbeiten.

Schritt 3 — Hypothesen nacheinander eliminiert (A/B-Läufe, Seed fixiert). Drei Komponenten des Trainings-Stacks wurden durch kontrollierte Vergleichsläufe mit identischem Seed geprüft:

- *Wand-/Loop-Penalties (Maßnahme B7)?* Entlastet ohne neuen Lauf: Die Batch-Validierung lief bereits auf der penalty-freien v11-Umgebung und lernte.
- *Erfolgs-Swarm (Maßnahme B8)?* Ein `--no-swarm`-Arm verlief deckungsgleich mit dem Kontrollarm — entlastet.
- *Streaming-Wrapper des Live-Monitorings?* Code-Inspektion (rein lesende Zugriffe) und ein Lauf ohne Wrapper: ebenfalls deckungsgleich — entlastet. (Nebenbefund behoben: Der Wrapper wurde bisher auch bei deaktivierter Live Map angelegt.)

Parallel dazu wurde die Batch-Validierung vom 06.07. *exakt reproduziert* (nackter Recurrent-PPO ohne Curriculum-Stack, 150k Schritte, batch=64): Sie erreichte am Endpunkt fast dieselben Werte wie damals (48 %/22 % vs. 52 %/22 %) — aber der *Verlauf* dazwischen oszillierte

wild (74 → 44 → 66 → 24 → 50 %, Abbildung 10 oben). Zusätzlich wurde ausgeschlossen, dass sich Curriculum-Pfad und nackter Pfad überhaupt funktional unterscheiden: Gewichtsinitialisierung, Weltseed-Folgen und die ersten 8192 Trainingsschritte (Rewards, Aktionen, Values) sind zwischen beiden *bit-identisch*.

Schritt 4 — Diagnose: chaotische Lerndynamik, nicht defekter Code. Die Gesamtschau aller Läufe ergab: Mit batch=64 ist die Lerndynamik hochvolatil (SR-Sprünge zwischen 10 und 74 % ohne Konvergenz), der Critic erreicht nur vereinzelte, nicht anhaltende `explained_variance`-Hochphasen. Die „Validierung“ der Batch-Größe am 06.07. hatte schlicht einen günstigen Schnappschuss dieses Prozesses erwischt — ein Einzel-Messpunkt einer volatilen Kurve. *Lehre: Hyperparameter-Entscheidungen brauchen Eval-Kurven, keine Endpunkte.*

Schritt 5 — Hypothesentest Batch-Größe: bestätigt. Der einzige verbliebene Unterschied zum funktionierenden v2/v10-Rezept war `batch_size`=64 statt 8 (bei batch=8 erfolgen pro Rollout ≈ 8-mal mehr Gradientenschritte auf denselben Daten; die v10-Reproduktion hatte damit EV bis 0.94 erreicht). Ein ansonsten identischer Lauf mit batch=8 zeigte ein qualitativ anderes Bild (Abbildung 10): stetiger Anstieg auf **84–88 % stochastische SR** bei 125–150 k Schritten — die besten Phase-1-Werte des Projekts — und erstmals über **50 % deterministische SR** in Phase 1 (batch=64: nie über 26 %). Ab ≈ 100 k Schritten hält der Critic anhaltende EV-Hochphasen (26 % der Iterationen > 0.5, gegenüber 3 % bei batch=64). Konsequenz: `batch_size` wurde auf 8 zurückgesetzt; der Geschwindigkeitsvorteil von batch=64 (Faktor 2,1) war mit instabilem Lernen erkaufte und damit wertlos.

Ergebnis der Etappe. Die v11-Stagnation hatte zwei unabhängige, sich überlagernde Ursachen: ein *Messartefakt* (Eval-Caps) und eine *echte Regression* (batch=64). Trainings-Stack, Seed-Replay, Wrapper und Umgebungsrevision wurden dabei experimentell entlastet — ein für die weitere Arbeit wertvolles Nebenergebnis, da die v11-Umgebung damit als methodisch sauber bestätigt ist. Die Konfiguration für die finalen Mehrfach-Läufe steht damit fest (v11-Umgebung, batch=8, Cap-4000-Evals, kurvenbasierte Bewertung).

9.7 Zeitleiste und Lessons Learned

Lessons Learned (negative Ergebnisse mit Erkenntniswert):

- *Visited-Mask in der Observation* destabilisiert den Argmax (jede 1-Bit-Änderung kann die Richtungspräferenz kippen).
- *Naiver Curriculum-Transfer* zerstört die Vorgänger-Politik — Mixed-Distribution bzw. leistungsbasierte Phasenwechsel sind nötig.
- *Feature-Kodierung schlägt Feature-Auswahl*: Der BFS-Gradient war nie „zu schwach“ — er war falsch kodiert (Delta-Fix: Signal 32× stärker).
- *Straf-Stacking* (Wand-, Loop-, Stagnations-Penalties) macht Stillstand lokal optimal; Information gehört in die Observation, nicht in den Reward.
- *Hyperparameter dominieren Features*: Alle v7-Feature-Ideen scheiterten an verstellten Basis-Hyperparametern; `ent_coef` war der entscheidende Faktor. Ohne das Changelog wäre die Root-Cause-Analyse (v2-Reverse-Engineering) nicht möglich gewesen.
- *Eval-Protokoll ist Teil des Experiments*: Data-Leakage über den Callback und das Gating auf einer signal-losen Metrik (det. SR bei hoher Entropie) verzerrten Selektion und Laufzeit unbemerkt.
- *Eval-Episoden-Caps verfälschen die Messung*: Dasselbe Modell misst sich bei Cap 600 als 48-%-, bei Cap 4000 als 86-%-Agent — eine als harmlos gedachte Beschleunigung machte das Phasenziel unerreichbar (Abschnitt 9.6).

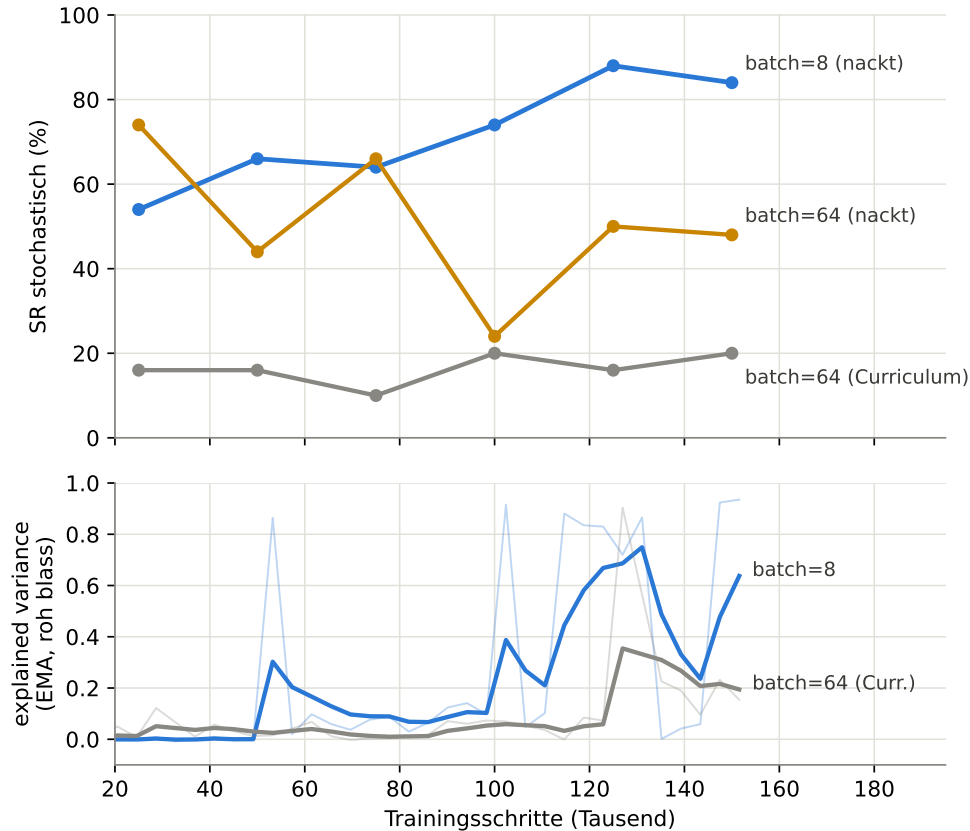


Abbildung 10: Batch-Größen-Forensik (alle Läufe Seed 1, Phase-1-Setup, Evals auf 50 Val-Seeds mit Cap 4000). Oben: stochastische SR — batch=8 steigt stetig auf 84–88 %, während batch=64 sowohl nackt (A1-Reproduktion) als auch im Curriculum-Stack chaotisch oszilliert bzw. stagniert. Unten: `explained_variance` des Critics (EMA-geglättet, Rohkurven bloss) — nur mit batch=8 entstehen ab $\approx 100k$ anhaltende Hochphasen. Erzeugt mit `scripts/plot_etappe6_figures.py`.

- *Einzel-Snapshots sind keine Validierung:* Die batch=64-Freigabe beruhte auf einem Messpunkt einer chaotisch oszillierenden Kurve; die exakte Reproduktion traf den Endpunkt, entlarvte aber den Verlauf. Hyperparameter-Entscheidungen nur auf ganzen Eval-Kurven.
- *Negative Kontrollen lohnen sich:* Der bit-genaue Vergleich von Curriculum-Pfad und nacktem Trainingspfad schloss eine ganze Verdächtigen-Klasse (Stack-Bugs) in Minuten aus und lenkte die Suche auf die Hyperparameter.

10 Umgebungsrevision v11 (06.07.2026)

Eine systematische Überprüfung der Umgebung gegen Best Practices des Environment- und Reward-Designs führte zu sieben verifizierten Änderungen:

1. **Tote Features entfernt:** Energie und Inventar waren konstant; Observation 231 $\rightarrow$ 229 Dimensionen (Tabelle 1).
2. **Straf-Stacking entschärft:** Wand-Penalty entfernt, Loop-Penalty $-0.15 \rightarrow -0.05$ (Tabelle 2).
3. **Exit-Platzierung nach BFS-Laufweg:** Zuvor wurden Kandidaten nach *Luftlinie* gewählt — der reale Laufweg bei nominell „35–45“ streute auf 42–75 Tiles ($\varnothing$ 55). Nach

Datum	Meilenstein	Ergebnis (Testset A)
12.05.	Spiel-Engine & Weltgenerierung spielbar	—
16.05.	DQN-Fehlstart → PPO; BFS-Kraftfeld-Bug gefixt	Oracle 0/3 → 3/3
16.05.	Curriculum P1; naiver Transfer kollabiert → Mixed-Dist.	56 % → 86 % stoch
16.05.	Phase 4 (Reward-Tuning), 3-Läufe-Protokoll	98 % stoch / 2 % det
18.05.	Delta-BFS-Kodierung + Temperature-Sampling	100 % stoch / 42 % det
07.06.	RecurrentPPO ohne BFS-Orakel (Kernbeitrag)	86 % / 36 %
08.06.	POMDP-These; Callback-Bugs; Swarm/Live-Map	—
11.06.	Data-Leakage behoben (Val-Set 6000er); Lösbarkeit 100 %	—
13.–15.06.	v6–v9 scheitern → Root-Cause (ent_coef/batch/n_steps)	0–12 %
25.06.	v10-Reproduktion abgeschlossen (Referenzmodell)	86 % (Val)
06.07.	Env-Revision v11 (10 Änderungen); batch=64 „validiert“ (am 07.07. widerlegt)	52 % stoch @150k (P1)
07.07.	v11-Läufe Seed 0/1 stagnieren; Monitoring-Redesign; Swarm-Analyse (B8) + Pool-Fix	16–28 % stoch (P1, Val)
07.07.	Forensik (Etappe 6): Eval-Cap-Artefakt gefixt; Swarm/Wrapper/Stack entlastet; batch 64 → 8	84–88 % stoch / 52 % det (P1, Val)
offen	≥3 volle Curriculum-Läufe (batch=8, Seeds 1–3), Det-Gap-Maßnahmen (P3)	—

Tabelle 8: Zeitleiste der RL-Entwicklung (Auswahl; vollständig in `CHANGELOG.md`). SR-Angaben je nach Etappe mit unterschiedlichem Protokoll — Werte vor dem 11.06. ohne strikte Val/Test-Trennung.

dem Fix liegen alle 150 Eval-Seeds exakt im Sollbereich (Abbildung 11); das Curriculum steuert die Schwierigkeit damit erstmals präzise.

- Prozess-globales Config-Leck behoben:** Die Weltgenerierungs-Konfiguration ist im C++-Kern global; parallel existierende Umgebungen überschrieben sich gegenseitig (u. a. verschob der Eval-Callback die Phase-3-Trainingsverteilung unbemerkt). Jede Umgebung stempelt ihre Konfiguration nun bei jedem Reset neu.
- Mining/Bauen/Kampf aus dem RL-Pfad entfernt** (Binding erlaubt nur noch Aktionen 0–3; tote Reward-Terme gestrichen).
- Entropie-Annealing korrigiert:** stetig 0.05 → 0.001 statt Sprung 0.05 → 0.01 beim Phasenwechsel.
- Eval-Callback:** phasengerechtes Gating (Tabelle 3), Episoden-Caps pro Phase, Det+Stoch-Doppelmessung.

Alle Änderungen wurden einzeln verifiziert (Lösbarkeit 150/150, Oracle-Agent 10/10 mit exakt BFS-Distanz Schritten, Obs-Layout-Konsistenz, Ablehnung unzulässiger Aktionen). Da sich die Aufgabenverteilung dadurch ändert, sind Ergebnisse ab v11 als neue Baseline zu behandeln.

11 Performance-Analyse: Training beschleunigen

11.1 CPU schlägt Apple-GPU (MPS)

Ein kontrollierter Benchmark (identisches Trainings-Setup, 8192 Steps) zeigt: Die GPU ist für dieses kleine Netz (LSTM 256, 229 Inputs) in *jeder* Konfiguration langsamer als die CPU —

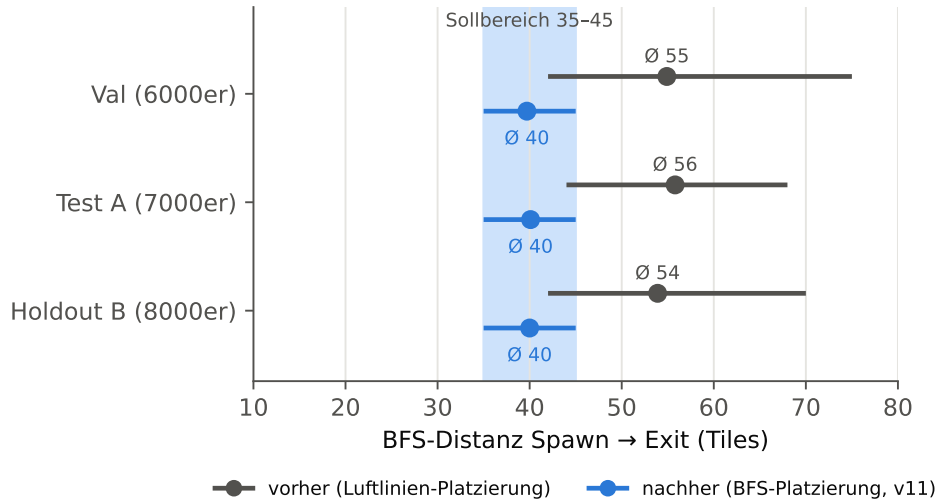


Abbildung 11: Wirkung der BFS-basierten Exit-Platzierung: tatsächliche Laufweg-Distanzen (min/Ø/max über je 50 Seeds) vor und nach dem Fix.

der Kernel-Dispatch-Overhead pro Mini-Update übersteigt den Rechengewinn (Abbildung 12). Dies deckt sich mit der Empfehlung der SB3-Autoren, GPUs erst bei CNN-Politiken oder großen Netzen einzusetzen [10].

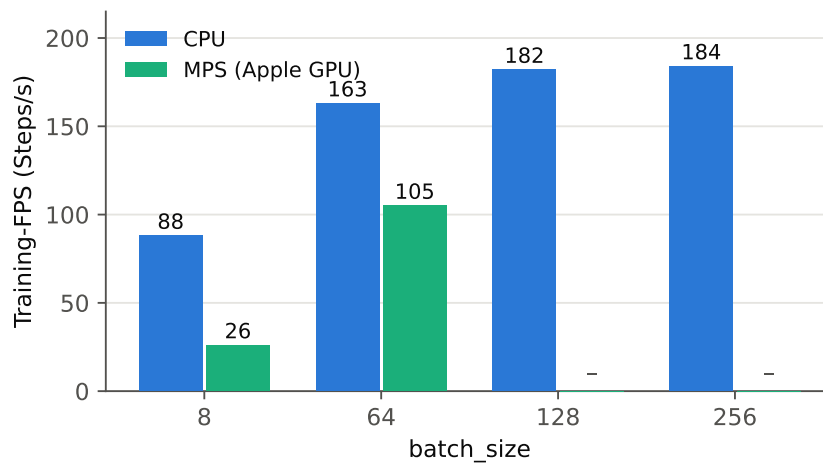


Abbildung 12: Training-FPS nach Device und Batch-Größe (RecurrentPPO, 16 Umgebungen, Apple M-Serie). MPS wurde für $\text{batch} \geq 128$ nicht weiter gemessen, da bereits bei 64 unterlegen.

11.2 Batch-Größe: vermeintliche Validierung (06.07.) — am Folgetag widerlegt

Mit `batch_size=8` fallen pro Rollout 5120 Gradient-Updates an — mehr Optimizer-Schritte als Umgebungsschritte. Da die Root-Cause-Analyse (Tabelle 7) die Batch-Größe nie isoliert getestet hatte, wurde ein Validierungslauf durchgeführt: 150k Phase-1-Steps mit `batch_size=64` (Abbildung 13). Ergebnis: gesunde Lernmetriken (`approx_kl` 0.01–0.06, `explained_variance` bis 0.72) und **52 % stochastische / 22 % deterministische SR** nach nur 150k Steps — bei **187 statt 88 FPS**. `batch_size=64` wurde daraufhin übernommen.

Diese Schlussfolgerung hielt der Überprüfung nicht stand. Die Forensik vom 07.07.

(Abschnitt 9.6) zeigte per exakter Reproduktion, dass der Validierungslauf ein günstiger Einzel-Schnappschuss einer chaotisch oszillierenden Lerndynamik war: Der Endpunkt (52 %) war reproduzierbar, der Verlauf dorthin sprang jedoch regellos zwischen 24 und 74 %, und in vollen Curriculum-Läufen stagnierte `batch=64` bei 16–28 %. Mit `batch_size=8` lernt dieselbe Konfiguration stetig auf 84–88 % (Abbildung 10). Die Batch-Größe *war* also doch ein kritischer v2-Erfolgsfaktor; sie wurde auf 8 zurückgesetzt. Der Abschnitt bleibt als Beleg des methodischen Fehlers erhalten: *Einzel-Messpunkte volatiler Prozesse validieren nichts*.

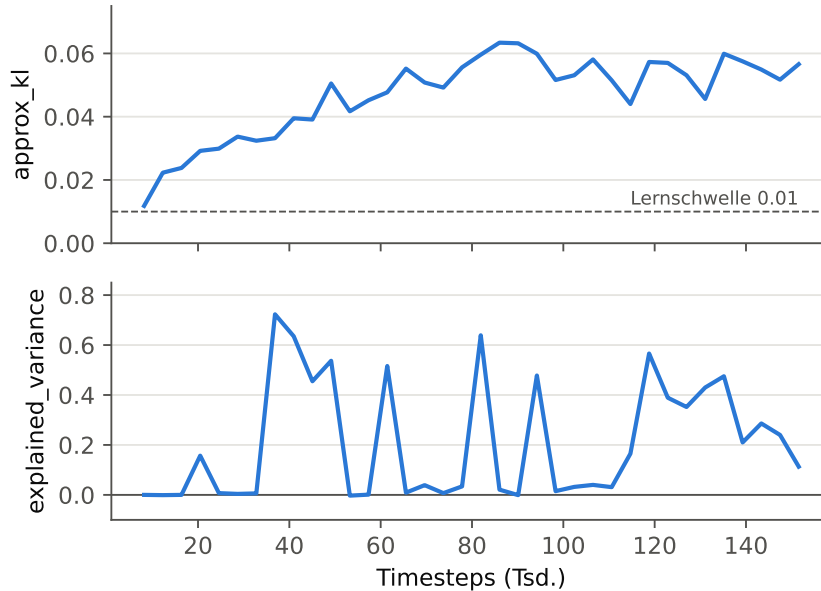


Abbildung 13: Validierungslauf `batch_size=64` (Phase-1-Setup, Env v11), Stand 06.07.: `approx_kl` über der Lernschwelle, `explained_variance` mit wiederkehrenden Hochphasen. Die daraus gezogene Freigabe von `batch=64` wurde am 07.07. widerlegt (Abschnitt 9.6): Die Hochphasen sind transient, die Dynamik konvergiert nicht.

12 Einordnung in verwandte Arbeiten

Stoneforge liegt strukturell zwischen drei etablierten Benchmarks: MiniGrid (teilbeobachtbare Grid-Navigation, PPO+LSTM als Standard-Baseline) [15], Crafter (prozedural generiertes 2D-Survival-Spiel mit Mining und Crafting) [16] und Procgen (Generalisierung über prozedurale Level) [1]. Publierte Zero-Shot-Ergebnisse auf prozeduralen Labyrinthen liegen typischerweise bei 25–53 % Success Rate; die hier erreichten 86 % (Test) bzw. 68 % (Holdout) bei nur 2,2 Mio. Trainingsschritten sind in diesem Kontext ein starkes Ergebnis. Für die Reduktion des Train/Test-Gaps ist Prioritized Level Replay [14] der vielversprechendste nächste Baustein; der vorhandene Seed-Pool-Mechanismus ist eine vereinfachte Variante davon.

13 Fazit und Ausblick

Stand: Die Machbarkeit ist gezeigt, beide Zielkriterien sind (stochastisch) erfüllt, und die Umgebung ist nach der v11-Revision methodisch sauber (präzise Schwierigkeitssteuerung, policy-invariantes Shaping, kein Straf-Stacking, keine toten Features). Die Forensik der Etappe 6 hat die Trainingskonfiguration konsolidiert (`batch=8`, Cap-4000-Evals, kurvenbasierte Bewertung) und liefert mit 84–88 % stochastischer / 52 % deterministischer Phase-1-SR die besten Zwischenwerte des Projekts. **Offene Schritte:**

1. **≥ 3 volle Curriculum-Läufe** mit der konsolidierten Konfiguration (batch=8, Seeds 1–3) für Mittelwert $\pm$ Standardabweichung; anschließend finaler Eval auf Testset A (7000–7049) und Holdout B (8000–8049).
2. **Det/Stoch-Gap schließen:** Die erstmals hohe deterministische Phase-1-SR ($>50\%$ statt bisher $\leq 26\%$) legt nahe, dass der gesunde Critic auch das Phase-3-Annealing stabilisiert — zu prüfen im vollen Lauf. Ergänzend: (a) Temperatur-Sweep der Evaluation; (b) Self-Imitation [17].
3. **Seed-Replay:** Der Erfolgs-Swarm zeigte im A/B-Test (Abschnitt 9.6) keinen Phase-1-Effekt; ob PLR-Semantik [14] in den späteren Phasen hilft, bleibt offen.
4. **Falls Holdout B unter Ziel:** vollwertiges Prioritized Level Replay (TD-Error-gewichtet) [14].

Literatur

- [1] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International conference on machine learning*, pages 2048–2056. PMLR, 2020. <https://arxiv.org/abs/1912.01588>.
- [2] Sebastian Risi and Julian Togelius. Increasing generality in machine learning through procedural content generation. *Nature Machine Intelligence*, 2(8):428–436, 2020. <https://www.nature.com/articles/s42256-020-0208-5>.
- [3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018. <http://incompleteideas.net/book/the-book-2nd.html>.
- [4] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.
- [5] Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps. *arXiv preprint arXiv:1507.06527*, 2015. <https://arxiv.org/abs/1507.06527>.
- [6] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. <https://www.nature.com/articles/nature14236>.
- [7] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016. <https://arxiv.org/abs/1506.02438>.
- [8] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 1928–1937. PMLR, 2016. <https://arxiv.org/abs/1602.01783>.
- [9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. <https://arxiv.org/abs/1707.06347>.

- [10] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. <http://jmlr.org/papers/v22/20-1364.html>.
- [11] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [12] Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
- [13] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, pages 41–48, 2009. <https://dl.acm.org/doi/10.1145/1553374.1553380>.
- [14] Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *International Conference on Machine Learning (ICML)*, pages 4940–4950. PMLR, 2021. <https://arxiv.org/abs/2010.03934>.
- [15] Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, et al. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. *Advances in Neural Information Processing Systems*, 36, 2023. <https://arxiv.org/abs/2306.13831>.
- [16] Danijar Hafner. Benchmarking the spectrum of agent capabilities. *arXiv preprint arXiv:2109.06780*, 2021. <https://arxiv.org/abs/2109.06780>.
- [17] Junhyuk Oh, Yijie Guo, Satinder Singh, and Honglak Lee. Self-imitation learning. In *International Conference on Machine Learning (ICML)*, pages 3878–3887. PMLR, 2018. <https://arxiv.org/abs/1806.05635>.